

Estrutura de Dados

Balanceamento e Altura

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Preço da Auto-Organização

A Aula 03 mostrou que a ABB organiza sua própria forma por comparação de chave. A Aula 04 provou, com o percurso em-ordem, que essa forma preserva a ordenação das chaves. O que as duas aulas deixaram em aberto é o reverso do problema: a mesma regra que organiza o índice pode também degenerá-lo, dependendo da ordem de chegada dos pontos.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Fator de Balanceamento

O fator de balanceamento de um nó é a altura da sua subárvore direita menos a altura da sua subárvore esquerda, calculado sobre a mesma função altura já definida em `arvore.py`. Fator zero indica equilíbrio perfeito; um ou menos um ainda é aceitável. Quando o valor absoluto chega a dois, a árvore está desequilibrada o bastante para exigir correção.



Prof. Everaldo Graciliano Souza Ribeiro

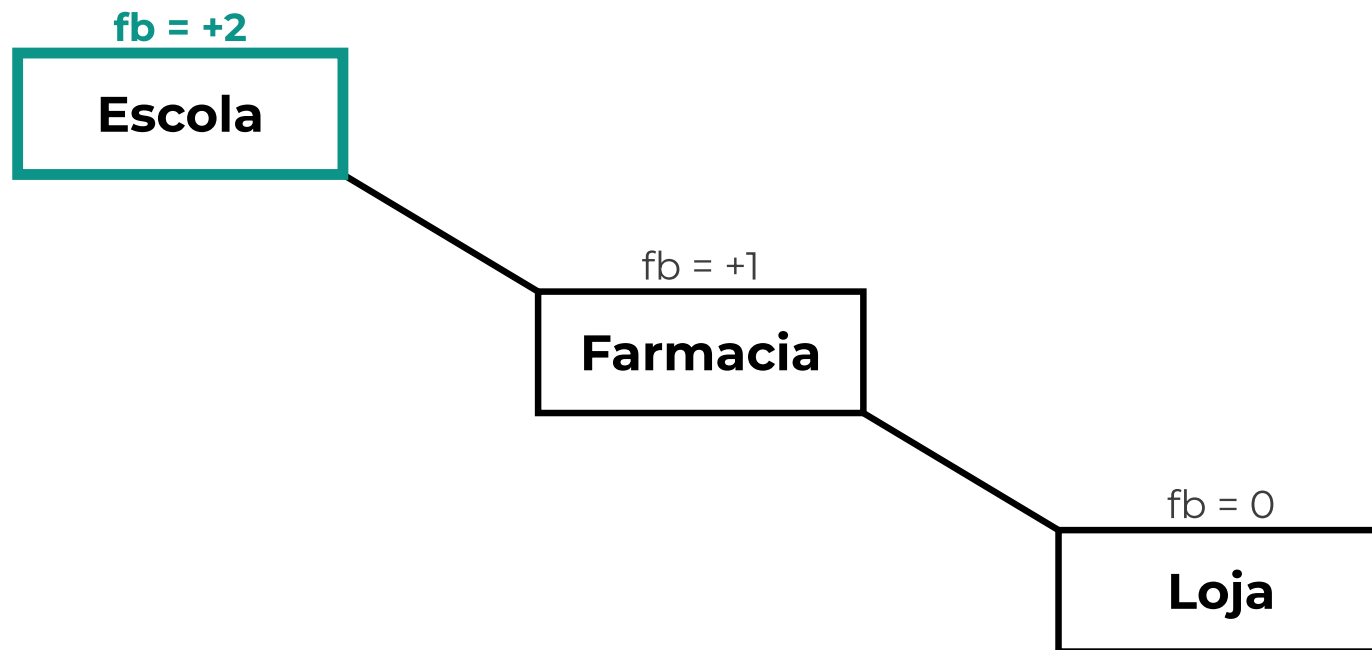
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Desequilíbrio Sobe pela Recursão



O desequilíbrio se acumula no nó mais alto do ramo: só quando **fb chega a 2** a rotação é disparada.



Fator de Balanceamento em Código

```
def fator_balanceamento(no):  
    """Altura da subarvore direita menos a da esquerda."""  
    # None nao desequilibra nada: fator zero por convencao.  
    if no is None:  
        return 0  
    return altura(no.direita) - altura(no.esquerda)
```

```
# fb = 0          → equilibrio perfeito  
# fb = 1 ou -1    → ainda aceitavel  
# fb = 2 ou -2    → gatilho da rotacao
```



A Rotação como Correção

Uma rotação simples reorganiza três ponteiros para corrigir o desequilíbrio sem violar a regra da ABB: o filho do lado mais pesado sobe para o lugar do nó desequilibrado, o próprio nó desce e vira filho desse antigo filho, e a subárvore que sobra entre os dois troca de lado. Nenhuma chave muda de posição relativa às demais — só a forma da árvore muda.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

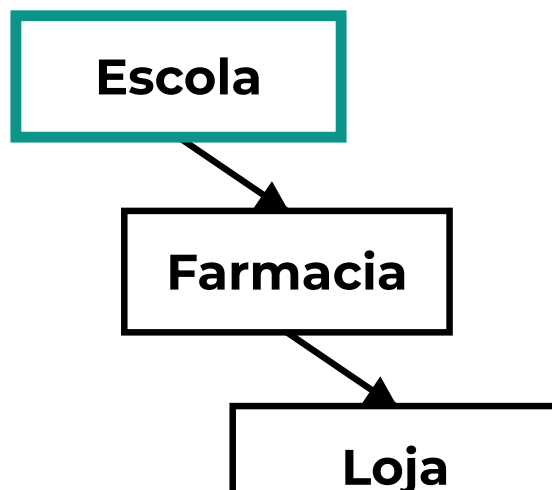
Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

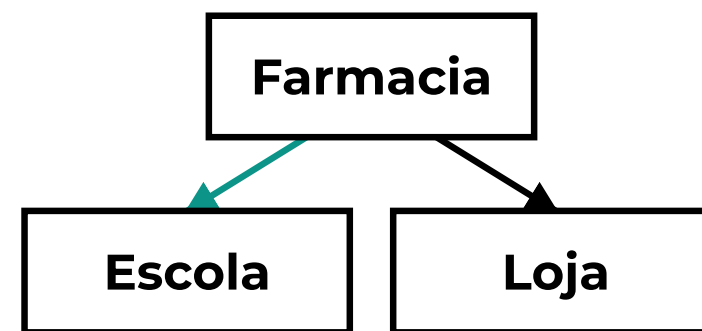
 <https://www.linkedin.com/in/everaldoribeiroads>

Rotação à Esquerda: Antes e Depois

antes — $fb(Escola) = +2$



depois — $rotacionar_esquerda(Escola)$



Dois ponteiros mudam: Escola perde a ligação com Farmacia, e **Farmacia passa a apontar para Escola** à esquerda — a ligação com Loja não muda.



Rotação em Código

```
def rotacionar_esquerda(no):  
    """Corrige desequilíbrio a direita: o filho direito sobe."""  
    novo_topo = no.direita  
    no.direita = novo_topo.esquerda  
    novo_topo.esquerda = no  
    return novo_topo  
  
# rotacionar_direita(no) e o espelho exato:  
# troca esquerda com direita e sobe o filho esquerdo.
```



Caso Reto × Caso Misto

Um nó pode desequilibrar de duas formas: reto, quando as duas inserções mais recentes seguiram para o mesmo lado, ou misto, quando a primeira desceu para um lado e a segunda virou para o outro. O caso reto se resolve com uma única rotação. O caso misto exige duas rotações em sequência e fica como desafio opcional desta aula.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Quando a Correção Acontece

`inserir_balanceado` segue exatamente os mesmos passos de `abb.inserir`, descendo por comparação de nome até encontrar uma subárvore vazia. A diferença aparece na volta da recursão: cada nó do caminho recalcula o próprio fator de balanceamento antes de devolver o resultado ao nível acima, rotacionando se necessário — a correção acontece de baixo para cima.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Inserção com Balanceamento

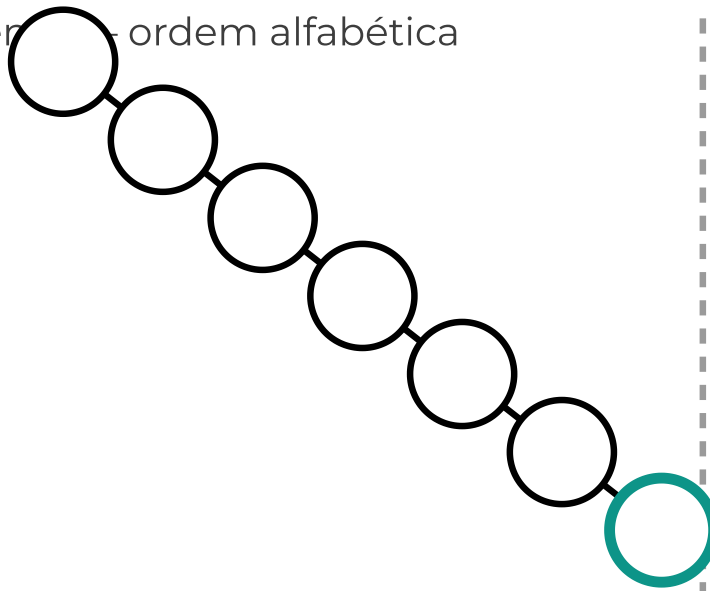
```
def inserir_balanceado(raiz, ponto):
    if raiz is None:
        return No(ponto)
    if ponto.nome < raiz.valor.nome:
        raiz.esquerda = inserir_balanceado(raiz.esquerda, ponto)
    else:
        raiz.direita = inserir_balanceado(raiz.direita, ponto)

    fb = fator_balanceamento(raiz)
    if fb == -2:                                # pesado a esquerda
        raiz = rotacionar_direita(raiz)
    elif fb == 2:                                # pesado a direita
        raiz = rotacionar_esquerda(raiz)
    return raiz
```



Mesmos 7 Pontos, Alturas Diferentes

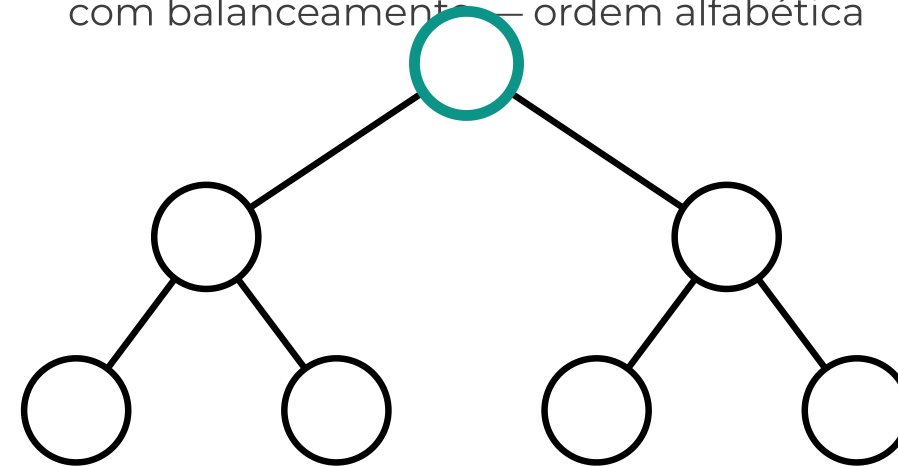
sem balanceamento — ordem alfabética



7 pontos — cadeia para a direita

altura 6 · 7 comparações no pior caso

com balanceamento — ordem alfabética



mesmos 7 pontos — árvore quase completa

altura 2 · 3 comparações no pior caso

Mesma ordem de inserção, mesmas 7 chaves — **a rotação reduz a altura de 6 para 2.**



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Que a Rotação Não Muda

A rotação altera exclusivamente a forma da árvore — nenhum ponto de entrega é criado, removido ou perde sua posição relativa na ordenação por nome. Um percurso em-ordem aplicado ao índice balanceado devolve exatamente a mesma sequência alfabética que o índice degenerado devolveria, na mesma ordem, apenas percorrendo um caminho muito mais curto para chegar a cada ponto.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Degenerado × Balanceado, Lado a Lado

```
ordem_alfabetica = sorted(PONTOS, key=lambda p: p.nome)

degenerado = construir_indice(ordem_alfabetica)
balanceado = construir_indice_balanceado(ordem_alfabetica)

print(altura(degenerado))      # 6
print(altura(balanceado))      # 2

# Mesmos 7 pontos, mesma ordem de insercao, duas arvores:
# so a estrategia de insercao muda a altura.
```



Atividades

Exercício 1

Por que o fator de balanceamento é calculado depois da chamada recursiva em `inserir_balanceado`, e não antes dela?

Exercício 2

Na rotação à esquerda, qual ponteiro deixa de existir e qual ponteiro passa a existir? O que acontece com a ligação que não muda?

Exercício 3

Por que inserir os 7 pontos em ordem alfabética, com balanceamento, produz altura 2 em vez da altura 6 da ABB comum?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Tratar o caso misto: quando o fator de balanceamento aponta para um lado mas o filho correspondente pesa para o lado oposto, aplicar uma rotação no filho antes de rotacionar o nó.
2. Escrever um teste que insira 15 pontos em ordem alfabética e confirme que a altura balanceada nunca ultrapassa 4.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>